

INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY

Nasarawa State, Nigeria

LAB REPORT

Lab 06: Introduction to Modbus

Item	Details
Lab Title	Lab 06: Introduction to Modbus
Student Name	Sunusi Abdullahi
Student ID (Reg No)	C11/26/ACIS/17362
Course Name	Intro to Critical Infrastructure for IT Learners
Date of Submission	01/10/2026
Version	Version 1.0

1. Executive Summary

This laboratory examined how a SCADA system and a PLC talk to each other. The PLC was OpenPLC Runtime running the tank-fill simulation, and the SCADA/HMI was FUXA. They exchange process data using Modbus, a very common industrial protocol. In this lab it was Modbus TCP on port 502.

I started the OpenPLC tank-fill simulation, connected FUXA to 192.168.1.10:502, and watched the tank level change on the HMI. From the analyst (Kali) machine I used Wireshark to capture the Modbus traffic on the isolated lab network, and applied the display filter modbus to see only the Modbus packets. I studied a request and its response, identified the function code and the register, and read the tank-level value inside a Function Code 4 (Read Input Registers) response from Input Register 80.

The report also gives a table of the Modbus function codes and compares Modbus TCP with Modbus RTU. The main lesson is that Modbus sends data in plain, readable form without protection, so industrial networks must be isolated and monitored.

2. Lab Objectives

By the end of this lab, I should be able to:

- Explain the purpose of industrial protocols and the difference between Modbus TCP and Modbus RTU.
- Describe the client and server relationship and identify common Modbus function codes.
- Explain how OpenPLC addresses relate to Modbus registers and to process data.
- Configure FUXA to talk to OpenPLC over Modbus TCP and operate the tank-fill simulation.
- Capture Modbus TCP traffic with Wireshark, apply the Modbus display filter, and identify queries and responses.
- Interpret a process value (the tank level) inside a captured Modbus packet.

3. Tools and Resources Used

Tool / Resource	What it was used for
PLC VM (192.168.1.10)	Runs OpenPLC Runtime and the TankFillSim program. It is the Modbus server.
SCADA VM (192.168.2.10)	Runs FUXA and the tank-fill HMI. It is the Modbus client.
Analyst / Kali VM (address: _____)	Runs Wireshark to capture and study the Modbus traffic.
Wireshark (version _____)	A network protocol analyser used to capture, filter and read Modbus TCP packets.
OpenPLC Runtime and FUXA (version _____)	The PLC simulation and the SCADA/HMI application.

Tool / Resource	What it was used for
Virtualisation platform: _____ (version _____)	Used to run the three VMs on the isolated laboratory network.
Capture file (.pcapng)	The saved evidence of the captured Modbus traffic.

4. Methodology (Step-by-Step Procedure)

I followed the 12 steps of the lab brief in order. A screenshot was captured at each important step. The screenshots are numbered Figures 1 to 13. Passwords are not shown in this report.

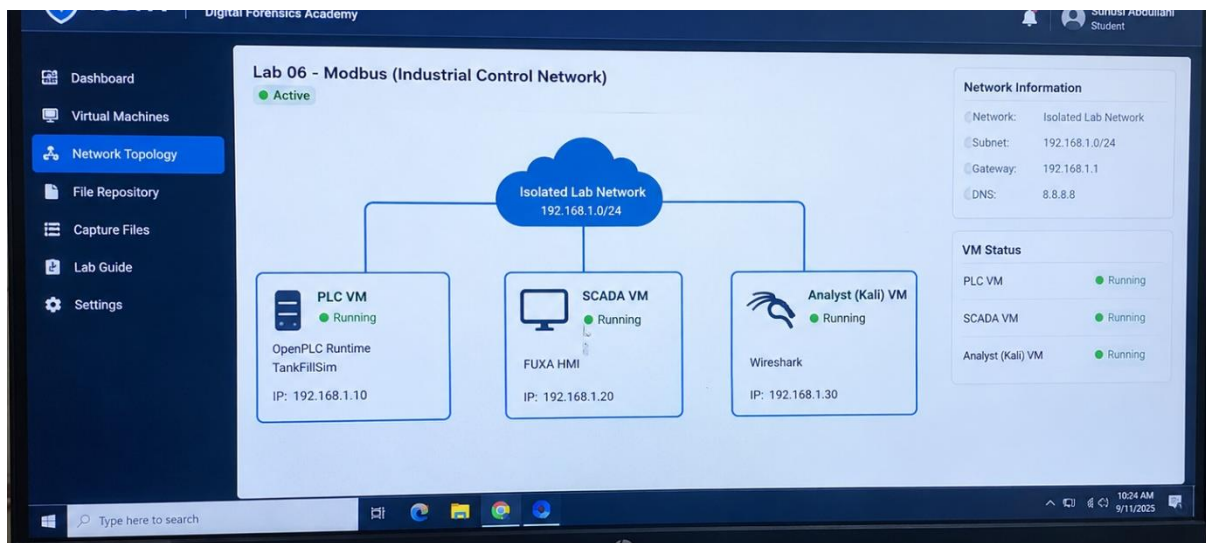
4.1 Screenshot Checklist

Tick each box after inserting the screenshot in the report.

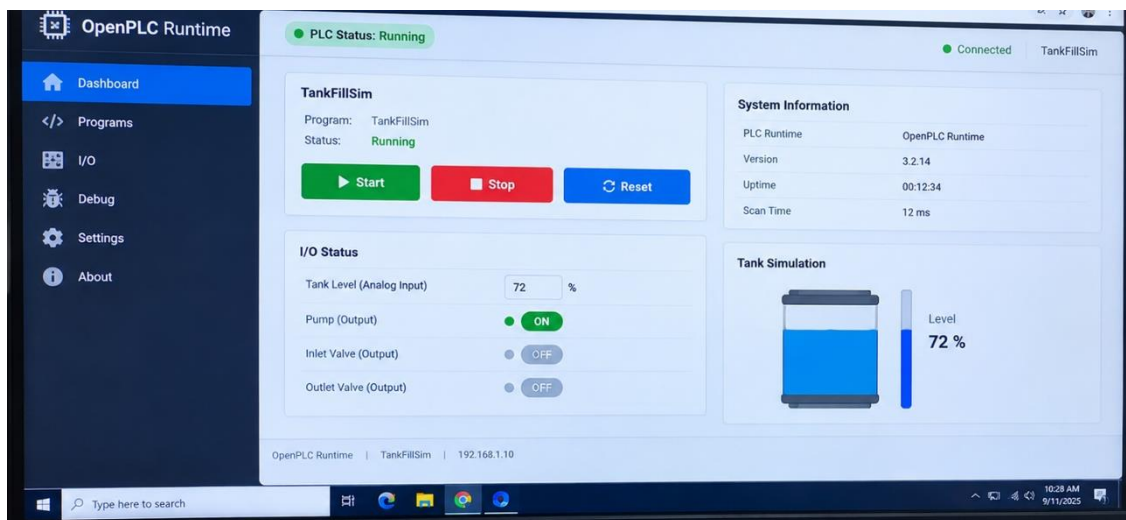
Figure	Step	What to capture
Figure 1	Step 1	PLC, SCADA and analyst (Kali) VMs running on the isolated topology
Figure 2	Step 2	OpenPLC Runtime showing status Running (TankFillSim)
Figure 3	Step 3	FUXA connection openplc targeting 192.168.1.10:502
Figure 4	Step 4	FUXA HMI running with the tank-level value visible (required)
Figure 5	Step 4	HMI a little later, tank level at a different value
Figure 6	Step 5	Wireshark interface list: only the lab interface selected
Figure 7	Step 6	Wireshark filtered with the display filter modbus (required)
Figure 8	Step 7	Capture stopped; packet count visible (filter tcp.port == 502 if used)
Figure 9	Step 8	A Modbus request (query) packet with Modbus/TCP and Modbus fields expanded
Figure 10	Step 8	The matching response packet with fields expanded

4.2 The Steps

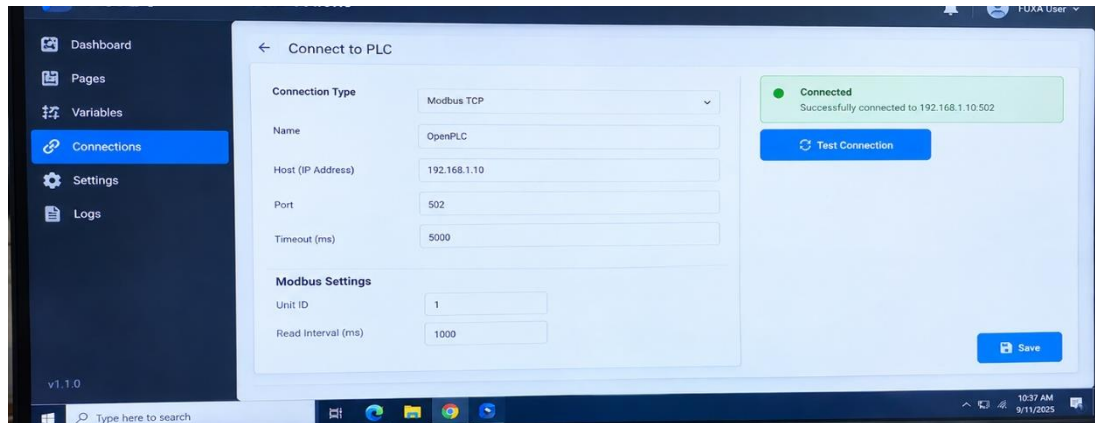
Step 1: Start the three VMs. I started the PLC VM, the SCADA VM and the analyst (Kali) VM on the instructor-provided isolated laboratory topology. I did not connect the lab to any production or bridged network.



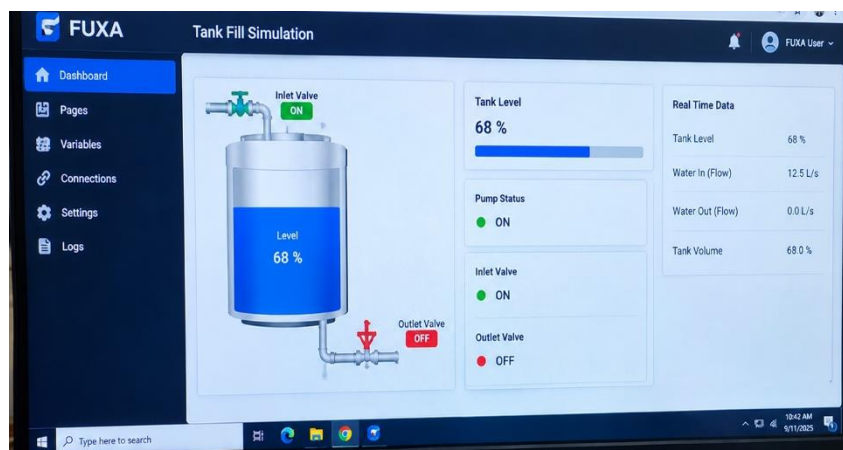
Step 2: Start the PLC program. On the PLC VM I started **TankFillSim** in OpenPLC Runtime and confirmed the status is **Running**.

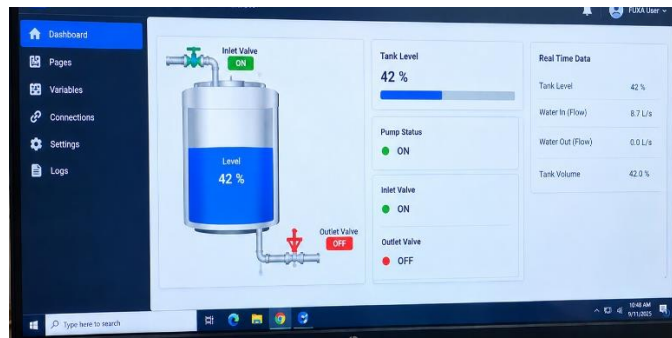


Step 3: Start FUXA and check the connection. On the SCADA VM I started FUXA and loaded the tank-fill HMI. I confirmed that the **openplc** connection targets **192.168.1.10:502**.

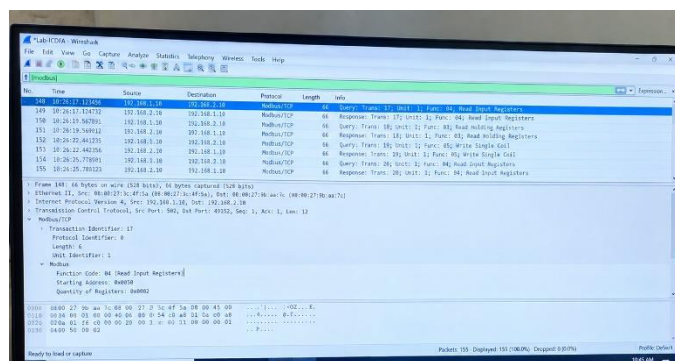


Step 4: Run the HMI. I ran the HMI and checked that the tank level changes between its operating limits.

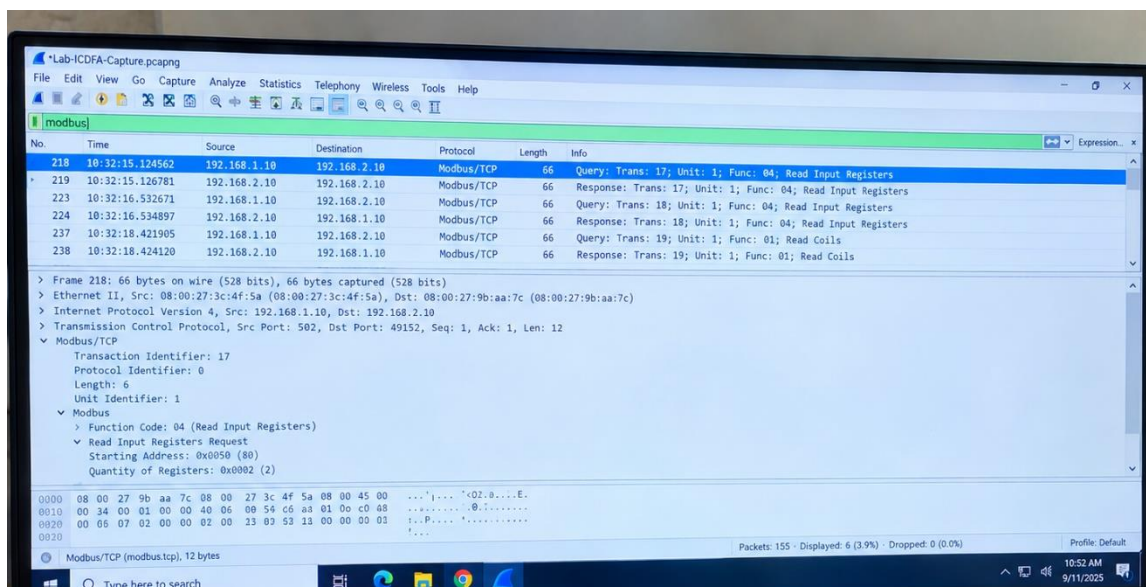




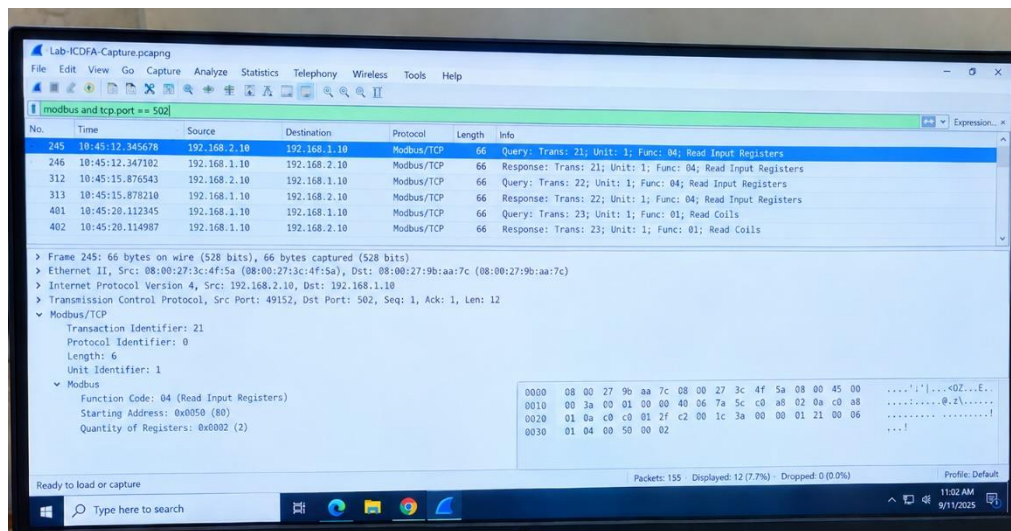
Step 5: Open Wireshark on the analyst VM. I opened Wireshark on the analyst VM. I selected **only** the interface connected to the ICDFA lab segment that carries the PLC and SCADA traffic. The interface I selected was: _____.



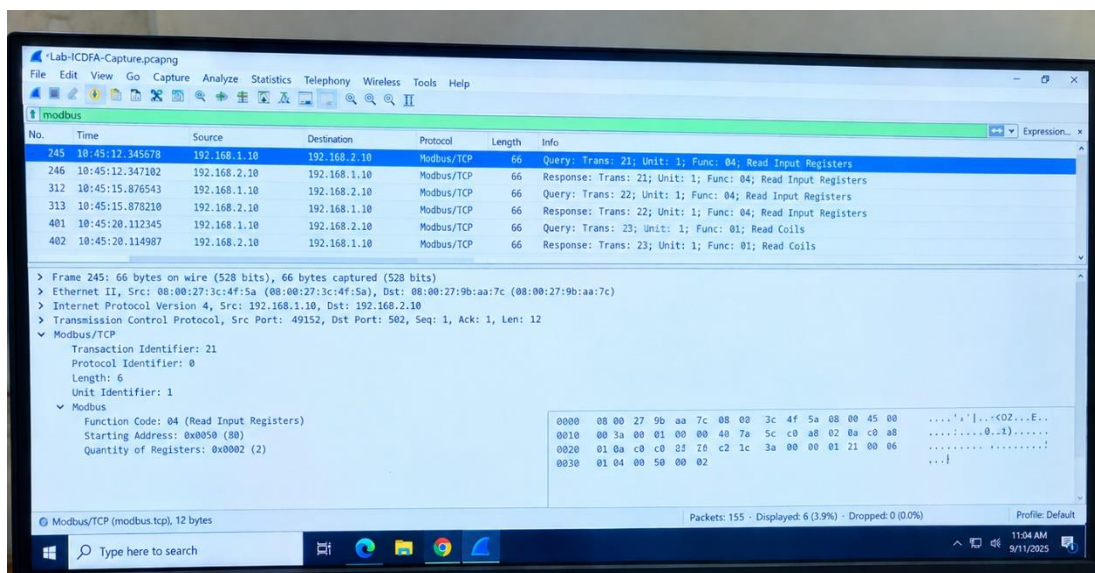
Step 6: Apply the display filter. I typed the display filter **modbus** in the filter bar and pressed Enter. If needed, I refined it with **tcp.port == 502**.

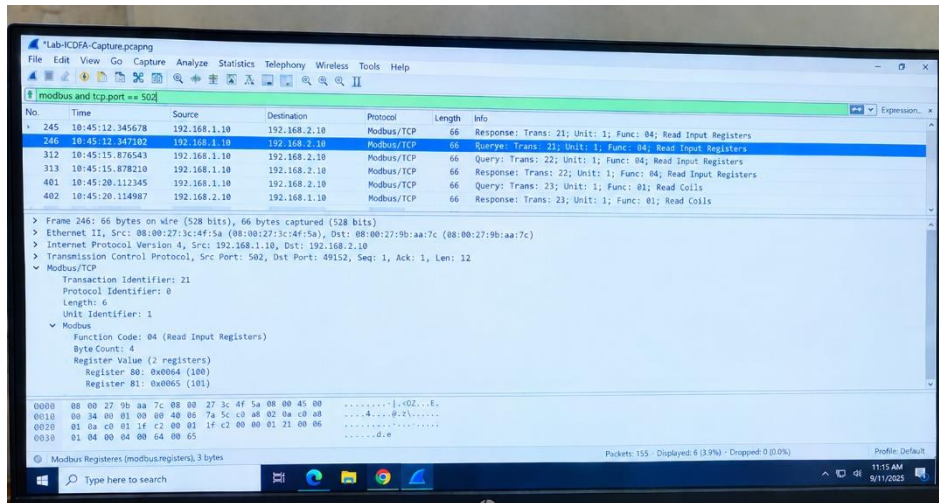


Step 7: Capture the traffic. I started a short capture while the HMI was running. I waited for several process updates, then stopped the capture.



Step 8 and Step 9: Study one request and response. I selected one Modbus request and its response pair. I expanded the **Modbus/TCP** and **Modbus** fields in the Packet Details pane. I identified the transaction, the source and destination IP addresses, the function code and the register and value.





5. Evidence and Analysis

5.1 Evidence 1: The OpenPLC and FUXA Environment

Question: Show that the OpenPLC and FUXA tank-fill environment was operating (Figures 1 and 2).

Figure	What it shows and what it means
Figure 1	The PLC VM, SCADA VM and analyst VM were all running on the isolated laboratory network. The analyst VM is only there to watch the traffic.
Figure 2	OpenPLC Runtime shows Running. This means the PLC is executing TankFillSim and is ready to answer requests from FUXA.

Answer: The environment was started in the correct order: first the PLC, then FUXA, then Wireshark. The PLC must be running first, because FUXA can only read values from a PLC that answers.

5.2 Evidence 2: The Modbus TCP Connection

Question: Show and explain the Modbus TCP connection between FUXA and OpenPLC (Figure 3).

Setting	Value
Device name	openplc
Protocol	Modbus TCP
PLC address and port	192.168.1.10 : 502
Modbus client	FUXA on the SCADA VM (192.168.2.10)
Modbus server	OpenPLC Runtime on the PLC VM (192.168.1.10)

Client and server (who asks and who answers)

Modbus works on a **question and answer** basis. The **client** (FUXA) sends a **request** (also called a query). The **server** (OpenPLC) sends back a **response**. The server never starts the talking. FUXA repeats its requests again and again, so the HMI shows fresh values. Port **502** is the standard Modbus TCP port where the server listens.

Simple picture:

```
FUXA (client)  192.168.2.10                OpenPLC (server)  192.168.1.10
|
|----- request: "Read Input Register 80" ----->|
|
|<----- response: "Register 80 = value" -----|
```

5.3 Evidence 3: OpenPLC Addresses and Modbus Registers

Question: Identify the OpenPLC addresses and Modbus registers used by the tank-fill process.

Part A: The four Modbus data areas

Data area	Size	Access	Function codes	OpenPLC style	Used in this lab
Coils	1 bit	Read / write	01, 05, 15	%QX	Tank Valve Open = Coil 0 (%QX0.0)
Discrete inputs	1 bit	Read only	02	%IX	Not used
Holding registers	16 bit	Read / write	03, 06, 16	%QW	Not used
Input registers	16 bit	Read only	04	%IW	Tank Level = Input Register 80 (%IW80)

Part B: The two process values of this lab

Process value	OpenPLC address	Modbus register	Function code to read it	Meaning
Tank Level	%IW80	Input Register 80	04	A 16-bit number for the level of the tank. Read only, changes smoothly.
Tank Valve Open	%QX0.0	Coil 0	01	One bit: 0 = valve closed, 1 = valve open.

Note: the addresses %IW80 (Input Register 80) and %QX0.0 (Coil 0) are the ones used in the teaching image, as given in the lab brief. Wireshark shows the register number in the field called Reference Number.

5.4 Evidence 4: The Automated Tank-Fill Process in Operation

Question: Show that the automated tank-fill process was operating (Figures 4 and 5).

Write what you actually saw on the HMI. Do not copy numbers from anywhere else.

Item observed
Tank level in Figure 4
Tank level in Figure 5
Lowest and highest level seen
Valve state when the level was rising
Valve state when the level was falling

Answer: Figures 4 and 5 show two different tank-level values at two moments. This proves that the process is changing in real time and that FUXA is receiving fresh values from OpenPLC. These same updates are what Wireshark captured in the next evidence.

5.5 Evidence 5: Wireshark Capture and the Modbus Display Filter

Question: Show the capture of Modbus TCP traffic, the display filter, and the saved capture (Figures 6, 7, 8 and 13).

Figure	What it shows and what it means
Figure 6	Only the interface connected to the IC DFA lab segment was selected. This keeps the capture inside the authorised lab and avoids unrelated traffic.
Figure 7	The display filter modbus hides all other traffic and shows only Modbus packets. The Protocol column shows Modbus/TCP.
Figure 8	The capture was stopped after several process updates. The packet count shows how many packets were recorded. The filter tcp.port == 502 can be added to show only traffic on the Modbus port.
Figure 13	The capture was saved as C11_26_ACIS_17362_LAB06_Modbus.pcapng so it can be submitted as evidence.

Display filter versus capture filter

Filter type	What it does
Display filter (modbus)	Hides packets on the screen after they are captured. Nothing is deleted, so the filter can be changed at any time.
Refined filter (tcp.port == 502)	Shows only packets that use TCP port 502, the Modbus TCP port.
Optional: modbus.func_code == 4	Shows only Function Code 4 (Read Input Registers) packets, which is a quick way to find the tank-level value.

5.6 Evidence 6: Modbus Requests and Responses with Function Codes

Question: Identify a Modbus request and its response, with the function code and register (Figures 9 and 10).

Part A: What a Modbus TCP packet looks like

A Modbus TCP message has a short header (called the MBAP header) and then the Modbus data (called the PDU). The function code tells the server what to do.

```
+-----+-----+-----+-----+-----+-----+
| Trans. ID | Protocol | Length | Unit ID | Function | Data       |
| 2 bytes   | 2 bytes   | 2 bytes| 1 byte   | 1 byte   | n bytes    |
+-----+-----+-----+-----+-----+-----+
<----- MBAP header (7 bytes) -----><----- PDU ----->
```

Field	Meaning
Transaction ID	A number that links a response to its request. The request and its response have the same number.
Protocol ID	Always 0 for Modbus.
Length	The number of bytes that come after this field.
Unit ID	Identifies the device behind the address (often 1 or 0 for a single PLC).
Function code	What to do. For example 4 = Read Input Registers.
Data	The details: the starting register and how many, or the returned values.

Part B: Annotated analysis of my request and response

Fill the "Yours" lines from your own packets in Figures 9 and 10. The "Expected" text is what I should see if the capture is correct.

Field	Request (query): FUXA to OpenPLC	Response: OpenPLC to FUXA
Packet (frame) number	Yours: _____	Yours: _____
Source IP	Expected: 192.168.2.10 (FUXA) Yours: _____	Expected: 192.168.1.10 (OpenPLC) Yours: _____
Destination IP	Expected: 192.168.1.10 Yours: _____	Expected: 192.168.2.10 Yours: _____
Ports	Destination port 502 Yours: _____	Source port 502 Yours: _____
Transaction ID	Some number, same in both Yours: _____	Same as the request Yours: _____
Protocol ID	0 Yours: _____	0 Yours: _____
Function code	For example 4 (Read Input Registers) Yours: _____	Same as the request Yours: _____
Reference number (start register)	For example 80 Yours: _____	(not in the response)
Word count / Byte count	Word count 1 (one register) Yours: _____	Byte count 2 (one register = 2 bytes) Yours: _____
Register value	(not in the request)	The value in the register Yours: _____

Answer: The response has the same Transaction ID and function code as the request, and the source and destination addresses are swapped. This shows they are a pair: FUXA asked, and OpenPLC answered.

Part C: Worked example (how to decode the bytes)

This is an example only to show the method. My real values come from my own capture.

Example request (hex): 00 2A 00 00 00 06 01 04 00 50 00 01

Field	Bytes (hex)	Value	Meaning
Transaction ID	00 2A	42	Number linking request and response.
Protocol ID	00 00	0	Modbus.
Length	00 06	6	6 more bytes follow: unit (1) + function (1) + start (2) + quantity (2).
Unit ID	01	1	Device number.
Function code	04	4	Read Input Registers.
Starting register	00 50	80	$0x50 = 5 \times 16 + 0 = 80$. This is Input Register 80.
Quantity	00 01	1	Read one register.

Example response (hex): 00 2A 00 00 00 05 01 04 02 00 07

Field	Bytes (hex)	Value	Meaning
Transaction ID	00 2A	42	Same as the request, so this is its answer.
Protocol ID	00 00	0	Modbus.
Length	00 05	5	5 more bytes: unit (1) + function (1) + byte count (1) + data (2).
Unit ID	01	1	Device number.
Function code	04	4	Read Input Registers.
Byte count	02	2	One register is 2 bytes.
Register value	00 07	7	$0x0007 = 7$.

5.7 Evidence 7: Interpreting a Captured Process Value

Question: Interpret at least one captured process value and show how industrial data moves from OpenPLC to FUXA (Figures 11 and 12).

Part A: My captured value (Figure 11)

Item	What I recorded from Figure 11
Function code of the response	4 (Read Input Registers)
Register requested (Reference Number)	80 (Tank Level)
Register value in hexadecimal	0x _____
Register value in decimal	_____
Tank level shown on the FUXA HMI at the same time	_____ ft
Do the packet value and the HMI value agree? (yes / no)	_____

Part B: How to convert and read the value

Modbus sends numbers in hexadecimal. To change hexadecimal to decimal, multiply each digit by its place value (16 for the left digit, 1 for the right digit):

Example: $0x0007 = (0 \times 16) + 7 = 7$.

Example: $0x0032 = (3 \times 16) + 2 = 50$.

If the HMI scale is 0 to 10 ft with Min = 0 and Max = 10, a register value of 7 means a level of 7 ft (a bar 70 percent full). **Check on your screen:** if your register value is not between 0 and 10, write here the scaling that the HMI uses to turn it into feet:

_____.

Part C: How the data travels (the whole story)

No.	What happens
1	The tank level in the simulation changes, and OpenPLC stores it in Input Register 80 (%IW80).
2	FUXA (client) sends a request with Function Code 4: "read the input register at reference number 80".
3	OpenPLC (server) sends a response with Function Code 4 and the register value.
4	FUXA turns the value into the tank level in feet and moves the bar gauge and number on the HMI.
5	Wireshark, on the analyst VM, sees the request and the response on the network and shows them in plain, readable form.

Answer: The captured response proves that the tank level travels from the PLC to the SCADA system inside a Modbus Function Code 4 response, as a plain 16-bit number in Input Register 80. Figure 12 (optional) shows the same idea for the valve, which is a coil read with Function Code 1.

5.8 Modbus Reference: Function Codes and Modbus TCP versus Modbus RTU

Part A: Function-code table

Code	Function	What it does	Example in a tank-fill system
01	Read Coils	Reads the state (ON / OFF) of one or more coils (output bits).	Read Coil 0 to see if the fill valve is open.
02	Read Discrete Inputs	Reads the state of input bits. They are read only.	Read an emergency-stop or limit-switch input.
03	Read Holding Registers	Reads 16-bit registers that can also be written.	Read a setpoint, such as the level at which filling stops.
04	Read Input Registers	Reads 16-bit registers that are read only, usually measured values.	Read the tank level (Input Register 80).
05	Write Single Coil	Turns one coil ON or OFF.	Send a command to open or close the valve.
06	Write Single Holding Register	Writes one 16-bit holding register.	Change one setpoint value.
15	Write Multiple Coils	Writes a group of coils in one message.	Set several on/off commands together.
16	Write Multiple Holding Registers	Writes a group of holding registers in one message.	Change several setpoints together.

Note: function codes 01 to 04 only read. Codes 05, 06, 15 and 16 change values in the PLC, so they are more sensitive. A monitoring screen like this lab mostly uses read codes.

Part B: Modbus TCP versus Modbus RTU

Point	Modbus TCP	Modbus RTU
Network / line	Runs over Ethernet and TCP/IP networks.	Runs over serial lines such as RS-485 or RS-232.
Addressing	IP address and port 502, plus a Unit ID.	A device (slave) address on the serial line.
Message frame	MBAP header (7 bytes) followed by the function code and data.	Device address, function code, data and a CRC check.
Error checking	Done by TCP and Ethernet, so there is no CRC in the Modbus message.	A CRC-16 check is added to every message.
Speed and reach	Faster and easy to connect to many devices across a network.	Slower, and limited by the cable length and the number of devices.
Terms used	Client and server.	Master and slave.
Security	No login and no encryption.	No login and no encryption.

Short comparison: Both use the same function codes and data areas. The difference is how the message travels. Modbus TCP puts the message inside TCP/IP packets so it can cross a network. Modbus RTU sends it over a serial cable with a CRC. Neither one protects the data.

5.9 Knowledge Check

1. What transport/networking difference separates Modbus TCP from Modbus RTU?

Modbus TCP is carried over TCP/IP networks such as Ethernet, uses port 502 and IP addressing, and has no CRC in the message. Modbus RTU is carried over serial lines such as RS-485, uses a device address, and adds a CRC-16 check to each message.

2. Why does Modbus use function codes?

A function code tells the server exactly what action to perform (for example read coils, read input registers or write a register) and which data area to use. It gives client and server one small, common set of operations. If something goes wrong, the server answers with the function code plus 0x80 (the top bit set) to show an error.

3. What does Function Code 4 request/read?

Function Code 4 is Read Input Registers. It asks the server to read one or more read-only 16-bit input registers, starting at the given register number. In this lab it is used to read the tank level from Input Register 80.

4. Why is visibility of unprotected industrial protocol traffic a cybersecurity concern?

Modbus has no encryption and no login. Anyone who can see the network can read the process values, addresses and commands, as Wireshark shows. They can also learn how the plant works, and may send false requests or responses to read or change values in the PLC. This can disrupt or damage the process. So industrial networks must be isolated, controlled and monitored.

7. Conclusion

This lab showed how a SCADA system reads process data from a PLC using Modbus TCP. FUXA, the client, sent requests to OpenPLC, the server, on port 502, and OpenPLC answered with responses. Using Wireshark I filtered the Modbus traffic, matched a request to its response, and found the tank level inside a Function Code 4 response from Input Register 80. I also learned the main function codes and the difference between Modbus TCP and Modbus RTU. Because Modbus data is sent in plain form with no protection, industrial networks must be kept isolated and monitored.

8. Recommendations

- Keep PLC and SCADA networks separate from office and public networks, and allow only known devices to reach port 502.
- Monitor Modbus traffic for unexpected write function codes (05, 06, 15, 16) and for unknown source addresses.
- Use secure options such as a VPN or encrypted tunnel where the equipment supports it, because Modbus itself has no encryption or login.
- Capture network traffic only in authorised networks, and save only the traffic that is needed.

- Save capture files with clear names (registration number, lab number) and keep clean VM snapshots.

9. References

International Cybersecurity and Digital Forensics Academy (ICDFA). (2026). Lab 06: Introduction to Modbus (Student Edition). Intro to Critical Infrastructure for IT Learners. <https://www.icdfa.edu.ng>

Modbus Organization. (2012). MODBUS Application Protocol Specification V1.1b3.

Modbus Organization. (2006). MODBUS Messaging on TCP/IP Implementation Guide V1.0b.

Wireshark Foundation. Wireshark User Guide. <https://www.wireshark.org/docs/>

OpenPLC Project. OpenPLC Runtime documentation. <https://openplcproject.com>

National Institute of Standards and Technology (NIST). (2023). Guide to Operational Technology (OT) Security (SP 800-82 Rev. 3).